# 💻 Infosys SE Elite Crack Vault — CampusMonk WhatsApp Premium Advanced Coding Problems (DSE/SP)

> **Target Role: Specialist Programmer (SP) & Digital Specialist Engineer (DSE)** (6.5 – 9.5 LPA) / Optional SE Bonus
> **Payload Architecture: Dynamic Programming, Greedy Algorithms & Custom Comparator Coding Problems**
> **Language Support:** Complete Production Implementations in **Java, C++, and Python**
> **Complexity:** Complete Time ($O(N)$) and Space ($O(N)$) Bounds with Edge-Case Analysis

---

## Problem 01: Minimum Coins Required (Coin Change) 🟡 `[OPTIONAL DSE / SP | HIGH PAYLOAD]`

> 🎯 **Exam Priority:** 🟡 `[OPTIONAL DSE / SP | HIGH PAYLOAD]`
> 📊 **Category:** `Dynamic Programming & Advanced Algorithms`
> **Question ID:** `CM_WA_CODING_Q01`

**Problem Description:** Given an array `coins[]` of size $N$ representing coin denominations and an integer target value $V$. Assuming an infinite supply of each coin denomination, determine the minimum number of coins needed to make change for $V$. If it is impossible, return `-1`.

- **Sample Input:** `coins = [25, 10, 5], V = 30`
- **Sample Output:** `2 (25 + 5 = 30)`

### Algorithmic Strategy & Optimal Formulation

Dynamic Programming (Bottom-Up Unbounded Knapsack). Let `dp[i]` be the min coins to form value `i`. Initialize `dp[0] = 0` and all other entries to $\infty$. For each amount $i$ from 1 to $V$ and each coin $c$: `dp[i] = min(dp[i], dp[i - c] + 1)`.

#### C++ Implementation

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int minCoins(vector<int>& coins, int V) {
    vector<int> dp(V + 1, 1e9);
    dp[0] = 0;
    for (int i = 1; i <= V; i++) {
        for (int c : coins) {
            if (i >= c) dp[i] = min(dp[i], dp[i - c] + 1);
        }
    }
    return dp[V] >= 1e9 ? -1 : dp[V];
}
```

#### Java Implementation

```java
import java.util.*;
public class MinCoins {
    public static int minCoins(int[] coins, int V) {
        int[] dp = new int[V + 1];
        Arrays.fill(dp, (int)1e9);
        dp[0] = 0;
        for (int i = 1; i <= V; i++) {
            for (int c : coins) {
                if (i >= c) dp[i] = Math.min(dp[i], dp[i - c] + 1);
            }
        }
        return dp[V] >= (int)1e9 ? -1 : dp[V];
    }
}
```

- **Time Complexity:** $O(N \times V)$
- **Space Complexity:** $O(V)$

---

### Problem 02: Subset Sum Problem 🟡 `[OPTIONAL DSE / SP | HIGH PAYLOAD]`

> 🎯 **Exam Priority:** 🟡 `[OPTIONAL DSE / SP | HIGH PAYLOAD]`
> 📊 **Category:** `Dynamic Programming & Advanced Algorithms`
> **Question ID:** `CM_WA_CODING_Q02`

**Problem Description:** Given an array `arr[]` of positive integers and a target sum $K$, determine if there exists a subset of the array with a sum equal to $K$.

- **Sample Input:** `arr = [3, 34, 4, 12, 5, 2], K = 9`
- **Sample Output:** `true (4 + 5 = 9)`

#### Algorithmic Strategy & Optimal Formulation

0/1 Knapsack Dynamic Programming. Maintain a boolean table `dp[i]` indicating whether sum `i` is achievable. Traverse backwards from $K$ down to `arr[j]` to optimize space to $O(K)$.

**C++ Implementation**

```cpp
#include <iostream>
#include <vector>
using namespace std;

bool isSubsetSum(vector<int>& arr, int K) {
    vector<bool> dp(K + 1, false);
    dp[0] = true;
    for (int x : arr) {
        for (int j = K; j >= x; j--) {
            dp[j] = dp[j] || dp[j - x];
        }
    }
    return dp[K];
}
```

**Java Implementation**

```java
import java.util.*;
public class SubsetSum {
    public static boolean isSubsetSum(int[] arr, int K) {
        boolean[] dp = new boolean[K + 1];
        dp[0] = true;
        for (int x : arr) {
            for (int j = K; j >= x; j--) {
                dp[j] = dp[j] || dp[j - x];
            }
        }
        return dp[K];
    }
}
```

- **Time Complexity:** $O(N \times K)$
- **Space Complexity:** $O(K)$

---

### Problem 03: Largest Number Formed from Array 🟡 `[OPTIONAL DSE / SP | HIGH PAYLOAD]`

> 🎯 **Exam Priority:** 🟡 `[OPTIONAL DSE / SP | HIGH PAYLOAD]`
> 📊 **Category:** `Dynamic Programming & Advanced Algorithms`
> **Question ID:** `CM_WA_CODING_Q03`

**Problem Description:** Given a list of non-negative integers, arrange them such that they form the largest possible number.

- **Sample Input:** `nums = [3, 30, 34, 5, 9]`
- **Sample Output:** `"9534330"`

#### Algorithmic Strategy & Optimal Formulation

Custom Comparator Sorting. Convert all integers to strings. Sort strings using the comparison rule: string $A$ precedes string $B$ if $(B + A) < (A + B)$. Handle the edge case where the largest element is "0".

**C++ Implementation**

```cpp
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
using namespace std;

string largestNumber(vector<int>& nums) {
    vector<string> s_nums;
    for (int n : nums) s_nums.push_back(to_string(n));
    sort(s_nums.begin(), s_nums.end(), [](const string& a, const string& b) {
        return a + b > b + a;
    });
    if (s_nums[0] == "0") return "0";
    string res = "";
    for (const string& s : s_nums) res += s;
    return res;
}
```

**Java Implementation**

```java
import java.util.*;
public class LargestNumber {
    public static String largestNumber(int[] nums) {
        String[] s_nums = new String[nums.length];
        for (int i = 0; i < nums.length; i++) s_nums[i] = String.valueOf(nums[i]);
        Arrays.sort(s_nums, (a, b) -> (b + a).compareTo(a + b));
        if (s_nums[0].equals("0")) return "0";
        StringBuilder sb = new StringBuilder();
        for (String s : s_nums) sb.append(s);
        return sb.toString();
    }
}
```

- **Time Complexity:** $O(N \log N \times L)$
- **Space Complexity:** $O(N \times L)$

---